

Claude Codeで作る、正規表現エンジン

— Brzozowski 微分：たった2つの関数30行が実用エンジンになる —

Nextbeat Osaka Lab #1

2026年7月23日（木）・株式会社ネクストビート大阪オフィス

毎日使う正規表現の"中身"を、ライブラリ品質で自作する

本日の流れ

時間	内容
19:00-19:10	受付・自己紹介
19:10-19:20	イントロ・ReDoS実演
19:20-20:30	ハンズオン（70分）
20:30-20:50	振り返り・懇親会
20:50-21:00	片付け・撤収

理論（Brzozowski 微分）は独立資料 `derivative.md` にまとめてある。
このスライドは当日の進行専用。

事件: 標準の RegExp が固まる

```
const native = /^(a+)$/;  
native.test("a".repeat(n) + "X");
```

- n を増やすたびに時間がおおよそ**倍**になる（実演で体感）
- これが **ReDoS**（Regular expression Denial of Service）。実サービス障害の実例もある
- 対して今日作るエンジンは同じ入力を**ミリ秒未満**で返す
- `pnpm bench:redos <n>`（`n` は22→26→28 の順で。**30 を超えると** `--force` が必須）

なぜ固まるのか（1分で）

- `(a+)+` に長い `a` の列 + 末尾1文字を食わせると、標準 RegExp は「`a` の列を何グループに分けるか」を全探索する
- 分け方は **$2^{(n-1)}$ 通り**（ $n=28$ で約1.3億）。1文字増えるたび2倍
- これが**バックトラッキング型**の宿命

詳しい理屈・数式トレースは `derivative.md` へ。今日はこの後 **15分で設計を握り、AI に実装させる**。

今日のゴール

「基本設計さえ固めれば、AI（Claude Code / Codex）で"使えるもの"がぱっと立ち上がってしまう」体験を持ち帰る。

- 人間が握るのは**基本設計**（[SPEC.md](#)の規則表）
- コードはAIが書く。設計表とコードが1対1だから、貼れば降ってくる
- 核は `nullable` / `derivative` の**2関数・約30行**だけ

当日の流れ（ハンズオン70分）

時刻	所要	ステップ
19:20-19:25	5分	0. 同期チェック
19:25-19:40	15分	1. 設計を固める
19:40-19:55	15分	2. 山場①（コア実装）
19:55-20:03	8分	3. まず壊す（正規化なし）
20:03-20:15	12分	4. 正規化②で直す
20:15-20:23	8分	5. 実食（持参正規表現）
20:23-20:30	7分	6. 設計の境界＋クロージング

ステップ0: 同期チェック (19:20-19:25)

全員、 `start` ブランチで:

```
git branch --show-current    # → start
pnpm test                    # → 27件中22件が赤
```

失敗メッセージは全て `TODO(山場①): nullable(...) - SPEC.md の v(r) の表を見て実装してください` の形。

このメッセージが出ていれば正常。 出ない・違うエラーが出る人は挙手。

ステップ1: 設計を固める (19:25-19:40)

[SPEC.md](#) を上から:

1. 記号の読み方 ($\vee \cdot \partial c \cdot \wedge \cdot \vee$)
2. nullable $v(r)$ の表 (6行)
3. derivative $\partial c(r)$ の表 (6行) – ★接続則の v 項に注目
4. 手トレース: $\partial b(a?b)$ を入力 `"b"` で計算 (`derivative.md` 参照)
5. `(a+)+` の微分列が同じ式に戻っていく予告 (②への伏線)

このあとの規則表を**そのまま**プロンプトに貼る。

ステップ2: 山場① – コア生成 (19:40-19:55)

[AGENT PROMPTS.md](#) の①のプロンプトをそのままコピーして

Claude Code / Codex に投入する (`src/derivative.ts` を開いた状態で)。

- 生成待ちの2～4分は「コード読みタイム」: `derivative.ts` の switch と SPEC の表を指差し確認
- 完了したら `pnpm test` → **27件全緑**が目標

```
pnpm test
```

赤かったテストが緑になる瞬間——今日のカタルシス。

ステップ3: まず壊す (19:55-20:03)

正規化 (②) がまだ無い状態で:

```
pnpm bench:redos 16
```

→ 「状態爆発を検知」で安全に打ち切られる (12文字プローブがあるのでフリーズしない)。
なぜ正規化が要るかを体感するステップ。

- **n** は **16 のまま**。20以上・引数省略 (デフォルト26) は打たない
- [SPEC.md](#) の「正規化 (ACI)」節を2分で確認

ステップ4: 正規化②で直す (20:03-20:15)

[AGENT PROMPTS.md](#) ②のプロンプトを投入 (`src/normalize.ts` を開いた状態で) :

```
pnpm test:all      # 全緑が目標 (tests/linear.test.ts 含む)  
pnpm bench:linear  #  $O(n)$  の実証
```

`bench:linear` の出力: 入力10倍で時間ほぼ10倍、**状態数は一定**。
標準 RegExp が固まった同じパターンが、1000万文字でも一瞬。

ステップ5: 実食 (20:15-20:23)

持参した「一番複雑な正規表現」を食わせる:

```
pnpm cli '<pattern>' '<input>'  
pnpm cli --search '<pattern>' '<input>'
```

パターンは必ずシングルクォートで囲む（`[ ] ( ) + |` をシェルが先に食う）。

対応構文は次のスライドへ ↓

対応構文（持参正規表現の簡約ガイド）

対応: 接続 / 選択 `|` / 量化子 `* + ? {n} {n,} {n,m}` / グループ `()` /
文字クラス `[...]` `[^...]` `a-z` / ドット `.` / エスケープ `\d \w \s \n \t \r`

エラーになる（対処法もメッセージに出る）：

- `^` `$` – test() は元々全体一致。外す（部分一致は `--search`）
- `\b` `\D` など未対応エスケープ – 言い換える
- `\1` などの後方参照 – 非対応（次スライド参照）
- `(?:...)` `(?=...)` – `(?:...)` は `(...)` に置換すれば等価

`{n,m}` はそのまま動く。非貪欲 `*? +? ??` は付けても意味は変わらない。

ステップ6: 設計の境界 (20:23-20:30)

あえて入れないもの＝後方参照 `\1`

- `(.+) \1` は正則言語の表現力を超え、DFAでは原理的に表現できない
- だから入れない。そして**入れないからこそ**線形時間と ReDoS 耐性が保証される
- Google RE2 / Rust regex / Go regexp も同じ判断

「何を入れないかで品質が決まる」――本日の設計判断の核心。

持ち帰り課題

- ③ パーサを AI に再生成させる ([AGENT PROMPTS.md](#) ③)
`src/parser.ts` は完成品として渡してある。退避 → 再生成 → `pnpm test` で確認 → 失敗時は `git restore`
- ④ 交差 `r&s` ・ 補集合 `-r` の追加 (④)。 `canonicalKey` と `normalize.ts` への波及に注意
- ④-b 持参正規表現に対応させる: `^` `$` `\b` を実際に通るようパーサを拡張する

答え（完成版）は `main` ブランチ。

今日持ち帰ってほしいこと

1. 「基本設計（規則表）さえ固めれば、AI で実装がぱっと立ち上がる」という開発体験
2. `nullable` / `derivative` のたった30行が実用品質のエンジンになる驚き
3. 「何を入れないかで品質が決まる」という設計判断の感覚

ここから振り返り・懇親会へ。お疲れさまでした！